

Dokumentacja projektu – AnimalStay

1. Opis projektu

Nazwa: AnimalStay

Cel: Stworzenie aplikacji do zarządzania hotelem dla zwierząt

Krótki opis systemu:

AnimalStay to aplikacja webowa służąca do zarządzania hotelem dla zwierząt. System umożliwia pracownikom hotelu rejestrację właścicieli i ich zwierząt, tworzenie rezerwacji pokoi, śledzenie aktywnych pobytów oraz zarządzanie opieką nad przebywającymi zwierzętami (leki, spacery).

Aplikacja jest przeznaczona wyłącznie do użytku wewnętrznego przez pracowników hotelu. Dostęp chroniony jest tokenem JWT. System rozróżnia dwie role użytkowników: worker (pracownik) i admin (administrator).

2. Technologie

- Frontend: React,Vite,TypeScript
- Backend: Node.js,Express.js,Prisma,TypeScript
- Baza danych: MySql
- Testy: Jest, ts-jest, Vitest
- Dodatkowe: bcrypt,jsonwebtoken,cors,dotenv,nodemon,SASS/SCSS,React Testing Library,React Router DOM,axios

3. Struktura projektu

```
AnimalStay/  
├── backend/  
│   ├── src/  
│   │   ├── app.ts  
│   │   ├── server.ts  
│   │   └── config/  
│   │       └── prisma.ts
```

```
| | └─ controllers/
| | | └─ auth.controller.ts
| | | └─ owner.controller.ts
| | | └─ pet.controller.ts
| | | └─ room.controller.ts
| | | └─ service.controller.ts
| | | └─ stay.controller.ts
| | └─ middleware/
| | | └─ auth.middleware.ts
| | | └─ error.middleware.ts
| | └─ routes/
| | | └─ auth.routes.ts
| | | └─ owner.routes.ts
| | | └─ pet.routes.ts
| | | └─ room.routes.ts
| | | └─ service.routes.ts
| | | └─ stay.routes.ts
| | └─ services/
| | | └─ auth.service.ts
| | | └─ owner.service.ts
| | | └─ pet.service.ts
| | | └─ room.service.ts
| | | └─ service.service.ts
| | | └─ stay.service.ts
| | └─ types/
| | └─ tests/
| | | └─ auth.test.ts
| | | └─ pet.test.ts
| | | └─ stay.test.ts
| └─ prisma/
| | └─ schema.prisma
| └─ .env
| └─ package.json
| └─ nodemon.json
| └─ tsconfig.json
```

```
|
└─ frontend/
    └─ src/
        |   └─ api/
        |   |   └─ axios.ts
        |   |   └─ auth.ts
        |   |   └─ owners.ts
        |   |   └─ pets.ts
        |   |   └─ rooms.ts
        |   |   └─ services.ts
        |   |   └─ stays.ts
        |   └─ components/
        |   |   └─ Navbar.tsx
        |   |   └─ ProtectedRoute.tsx
        |   |   └─ DatePicker.tsx
        |   └─ img/
        |   └─ pages/
        |   |   └─ Home.tsx
        |   |   └─ Login.tsx
        |   |   └─ Dashboard.tsx
        |   |   └─ Pets.tsx
        |   |   └─ Reservations.tsx
        |   |   └─ RoomMap.tsx
        |   └─ styles/
        |   |   └─ index.scss
        |   |   └─ app.scss
        |   └─ tests/
        |   |   └─ Home.test.tsx
        |   |   └─ setup.ts
        |   └─ types/
        |   |   └─ Pet.ts
        |   |   └─ Stay.ts
        |   └─ App.tsx
    └─ public/
    └─ index.html
```

└─ vite.config.ts

└─ package.json

4. Funkcjonalności

- Logowanie i autoryzacja pracowników
- Zarządzanie właścicielami zwierząt (CRUD)
- Zarządzanie zwierzętami (CRUD)
- Rezerwacja pokoi z wyborem dat i usług dodatkowych
- Mapa pokoi z podglądem zajętości w wybranym przedziale dat
- Dashboard z listą aktywnych pobytów, alertami o lekach i spacerach
- Aktualizacja flag opieki (needsMeds, needsWalk) w czasie rzeczywistym

5. Schemat bazy danych

Baza danych: MySQL, nazwa: animalstay

ORM: Prisma 5

Modele:

User

id Int (PK, autoincrement)

email String (UNIQUE)

password String (zahashowane bcrypt)

role String (domyślnie: "worker")

Owner

id Int (PK, autoincrement)

name String

email String (UNIQUE)

phone String

pets Pet[] (relacja 1:N)

Pet

id Int (PK, autoincrement)
name String
species String (np. "dog", "cat", "bird")
breed String? (opcjonalne)
age Int
ownerId Int (FK -> Owner, onDelete: Cascade)
owner Owner
stays Stay[] (relacja 1:N)

Room

id Int (PK, autoincrement)
number Int
type String (Standard | Deluxe | Suite | Kennel | Aquarium)
pricePerDay Float
status String (available | occupied)
stays Stay[]

Stay

id Int (PK, autoincrement)
startDate DateTime
endDate DateTime
status String (scheduled | checked-in | checked-out)
notes String?
needsMeds Boolean (domyślnie: false)
needsWalk Boolean (domyślnie: true)
medsInfo String? (informacje o lekach)
petId Int (FK -> Pet, onDelete: Cascade)
roomId Int (FK -> Room)
services StayService[]
payment Payment?

Service

id Int (PK, autoincrement)

name String

price Float

stays StayService[]

StayService (tabela łącząca Stay i Service – relacja N:M)

stayId Int (FK -> Stay, onDelete: Cascade)

serviceId Int (FK -> Service)

PK: [stayId, serviceId]

Payment (

id Int (PK, autoincrement)

amount Float

status String (domyślnie: "pending")

paidAt DateTime? (opcjonalne)

stayId Int (UNIQUE, FK -> Stay)

Relacje:

Owner 1 --< N Pet

Pet 1 --< N Stay

Room 1 --< N Stay

Stay N >--< N Service (przez StayService)

Stay 1 --< 1 Payment

6. Jak uruchomić

Wymagania wstępne

- Node.js
- MySQL (uruchomiony lokalnie lub zdalnie)
- npm

KROK 1 – Konfiguracja bazy danych

Utwórz bazę danych MySQL o nazwie "animalstay":

```
CREATE DATABASE animalstay;
```

KROK 2 – Konfiguracja zmiennych środowiskowych backendu

Utwórz plik backend/.env o treści:

```
DATABASE_URL="mysql://root:<hasło>@localhost:3306/animalstay"
```

```
JWT_SECRET=animalstay_super_secret_key_2024
```

KROK 3 – Instalacja zależności backendu

```
cd backend
```

```
npm install
```

KROK 4 – Migracja bazy danych (Prisma)

```
npx prisma migrate dev --name init
```

lub jeśli migracje już istnieją:

```
npx prisma migrate deploy
```

KROK 5 – Uruchomienie backendu

```
npm start
```

Serwer uruchomi się na <http://localhost:5000>

Przy starcie automatycznie seedowane są pokoje i usługi.

KROK 6 – Instalacja zależności frontendu

```
cd frontend
```

```
npm install
```

KROK 7 – Uruchomienie frontendu

```
npm run dev
```

Frontend uruchomi się na <http://localhost:5173>

KROK 8 – Utworzenie pierwszego użytkownika

Wywołaj endpoint rejestracji (np. przez Postman lub curl):

POST <http://localhost:5000/api/auth/register>

Content-Type: application/json

```
{  
  "email": "admin@animalstay.com",  
  "password": "Admin1234",  
  "role": "admin"  
}
```

7. Testy

BACKEND – testy jednostkowe (Jest)

Lokalizacja: backend/src/tests/

Framework: Jest + ts-jest

Mockowanie: Prisma i biblioteki zewnętrzne mockowane przez jest.mock()

Uruchomienie wszystkich testów backendu:

```
cd backend
```

```
npm test
```

Uruchomienie pojedynczego pliku:

```
npx jest auth.test.ts
```

```
npx jest pet.test.ts
```

```
npx jest stay.test.ts
```

FRONTEND – testy komponentów (Vitest)

Lokalizacja: frontend/src/tests/

Framework: Vitest + React Testing Library

Środowisko: jsdom (konfiguracja w vite.config.ts)

Uruchomienie testów frontendu:

```
cd frontend
```

```
npm test
```

Plik testowy:

```
Home.test.tsx
```

8. Problemy/Co można poprawić

Brak edycji rekordów

Aplikacja umożliwia tworzenie i usuwanie właścicieli, zwierząt i rezerwacji, ale nie oferuje formularzy edycji istniejących rekordów.

Płatności niezaimplementowane

Model Payment istnieje w schemacie bazy danych, ale nie ma dedykowanych endpointów ani interfejsu do zarządzania płatnościami.

Brak odświeżania tokenu

Token JWT wygasa po 24h. Aplikacja nie obsługuje automatycznego odświeżania (refresh token). Po wygaśnięciu użytkownik jest wylogowywany i musi zalogować się ponownie.

ProtectedRoute bez weryfikacji ważności tokenu

Komponent ProtectedRoute sprawdza tylko czy token istnieje w localStorage, ale nie weryfikuje jego ważności po stronie klienta. Wygaśły token zostanie wykryty dopiero przy pierwszym żądaniu do API (przez Axios interceptor).